

✓ Prompt Ladder / Prompt Mastery — Gemini Hands-On Exercise

This lab demonstrates how to progressively control an LLM through structured prompting.

```
#Install Gemini SDK!
```

```
!pip install google-generativeai --upgrade
```

```
Requirement already satisfied: google-generativeai in /usr/local/lib/python3.12/dist-packages (0.8.6)
Requirement already satisfied: google-ai-generativelanguage==0.6.15 in /usr/local/lib/python3.12/dist-packages (from google-generativeai) (0.6.15)
Requirement already satisfied: google-api-core in /usr/local/lib/python3.12/dist-packages (from google-generativeai) (2.29.0)
Requirement already satisfied: google-api-python-client in /usr/local/lib/python3.12/dist-packages (from google-generativeai) (2.12.3)
Requirement already satisfied: google-auth>=2.15.0 in /usr/local/lib/python3.12/dist-packages (from google-generativeai) (2.47.0)
Requirement already satisfied: protobuf in /usr/local/lib/python3.12/dist-packages (from google-generativeai) (5.29.5)
Requirement already satisfied: pydantic in /usr/local/lib/python3.12/dist-packages (from google-generativeai) (2.12.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from google-generativeai) (4.67.1)
Requirement already satisfied: typing-extensions in /usr/local/lib/python3.12/dist-packages (from google-generativeai) (4.15.0)
Requirement already satisfied: proto-plus<2.0.0dev,>=1.22.3 in /usr/local/lib/python3.12/dist-packages (from google-ai-generativelanguage==0.6.15) (1.22.3)
Requirement already satisfied: googleapis-common-protos<2.0.0,>=1.56.2 in /usr/local/lib/python3.12/dist-packages (from google-api-core) (1.62.0)
Requirement already satisfied: requests<3.0.0,>=2.18.0 in /usr/local/lib/python3.12/dist-packages (from google-api-core) (2.31.0)
Requirement already satisfied: pyasn1-modules>=0.2.1 in /usr/local/lib/python3.12/dist-packages (from google-auth>=2.15.0->google-auth) (0.4.0)
Requirement already satisfied: rsa<5,>=3.1.4 in /usr/local/lib/python3.12/dist-packages (from google-auth>=2.15.0->google-auth) (4.9)
Requirement already satisfied: httplib2<1.0.0,>=0.19.0 in /usr/local/lib/python3.12/dist-packages (from google-api-python-client) (0.19.0)
Requirement already satisfied: google-auth-httplib2<1.0.0,>=0.2.0 in /usr/local/lib/python3.12/dist-packages (from google-api-python-client) (0.2.0)
Requirement already satisfied: uritemplate<5,>=3.0.1 in /usr/local/lib/python3.12/dist-packages (from google-api-python-client) (4.1.1)
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.12/dist-packages (from pydantic) (0.7.0)
Requirement already satisfied: pydantic-core==2.41.4 in /usr/local/lib/python3.12/dist-packages (from pydantic) (2.41.4)
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python3.12/dist-packages (from pydantic) (0.4.2)
Requirement already satisfied: grpcio<2.0.0,>=1.33.2 in /usr/local/lib/python3.12/dist-packages (from google-api-core[grpc]!=2.0.0) (1.62.0)
Requirement already satisfied: grpcio-status<2.0.0,>=1.33.2 in /usr/local/lib/python3.12/dist-packages (from google-api-core[grpc]!=2.0.0) (1.62.0)
Requirement already satisfied: pyparsing<4,>=3.1 in /usr/local/lib/python3.12/dist-packages (from httplib2<1.0.0,>=0.19.0->httplib2) (3.1.2)
Requirement already satisfied: pyasn1<0.7.0,>=0.6.1 in /usr/local/lib/python3.12/dist-packages (from pyasn1-modules>=0.2.1->pyasn1) (0.6.1)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.18.0) (3.3.2)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.18.0->requests) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.18.0->requests) (2.2.3)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.18.0->requests) (2025.1.1)
```

```
import google.generativeai as genai
print('Gemini SDK imported.')
```

```
Gemini SDK imported.
```

```
import google.generativeai as genai
```

```
API_KEY = 'AIzaSyBpFC7F5mUsfqe6zEFFkbXE1JRZdKS5qnk'
genai.configure(api_key=API_KEY)
print('API configured.')
```

```
API configured.
```

```
/usr/local/lib/python3.12/dist-packages/google/colab/_import_hooks/_hook_injector.py:55: FutureWarning:
```

```
All support for the `google.generativeai` package has ended. It will no longer be receiving
updates or bug fixes. Please switch to the `google.genai` package as soon as possible.
See README for more details:
```

```
https://github.com/google-gemini/deprecated-generative-ai-python/blob/main/README.md
```

```
loader.exec_module(module)
```

```
# Initialize model
model=genai.GenerativeModel('gemini-2.5-flash')
print('Model loaded: gemini-2.5-flash')
```

```
Model loaded: gemini-2.5-flash
```

✓ 1. Zero-Shot Prompting

```
model=genai.GenerativeModel('gemini-2.5-flash') # Ensure model is initialized
print('Model loaded: gemini-2.5-flash')
```

```
print('\n=== ZERO-SHOT ===')
prompt='Explain Neural Networks.' # Define the zero-shot prompt
```

```

print('Prompt:', prompt)
resp=model.generate_content(prompt) # Generate content using the defined prompt
print('Response:')
print(resp.text)

Model loaded: gemini-2.5-flash

=== ZERO-SHOT ===
Prompt: Explain Neural Networks.
Response:
Neural Networks are a subset of machine learning, inspired by the structure and function of the human brain. They are powerful c

Imagine a network of interconnected "neurons" (nodes) organized in layers, processing information much like our brain does.

### The Core Components:

1. Neurons (Nodes): These are the fundamental building blocks. Each neuron receives one or more inputs, processes them, and

2. Layers: Neurons are typically organized into layers:
    * Input Layer: Receives the raw data (features) you feed into the network. The number of neurons here equals the numbe
    * Hidden Layers: One or more layers between the input and output layers. These are where the network performs complex
    * Output Layer: Produces the final result of the network (e.g., a prediction, a classification). The number of neurons

3. Connections (Synapses) and Weights:
    * Neurons in one layer are connected to neurons in the next layer.
    * Each connection has an associated weight, which determines the strength or importance of that connection. Weights ar

4. Biases:
    * Each neuron (or layer) also has a bias term. A bias can be thought of as an adjustable threshold or offset that allo

5. Activation Function:
    * After a neuron receives its weighted sum of inputs (and adds its bias), it passes this result through an activation fu
    * Purpose:
        * Introduces Non-linearity: Without activation functions, a neural network would just be performing linear operati
        * Normalizes Output: It squashes the output of the neuron to a specific range (e.g., between 0 and 1, or -1 and 1)
    * Common Examples: Sigmoid, ReLU (Rectified Linear Unit), Tanh, Softmax (often used in the output layer for multi-clas

### How a Neural Network Works (The "Flow"):Forward Propagation:
    * Input: Data is fed into the input layer.
    * Weighted Sum: Each neuron in the first hidden layer takes the outputs from the input layer, multiplies them by their
    *  $\text{Weighted Sum} = (\text{Input1} * \text{Weight1}) + (\text{Input2} * \text{Weight2}) + \dots + \text{Bias}$ 
    * Activation: This weighted sum is then passed through the neuron's activation function to produce an output.
    * This process continues layer by layer until the output layer produces the final prediction.

2. Learning (Training): This is the magic!
    * Loss Function (Cost Function): After the network makes a prediction (forward propagation), a loss function measu
    * Backpropagation: This is the core algorithm for training neural networks.
        * The error calculated by the loss function is propagated backwards through the network.
        * During backpropagation, the network calculates the gradient of the loss function with respect to each weight and b
    * Optimizer (Gradient Descent): An optimizer (like Gradient Descent) then uses these gradients to update the weigh
        * The learning rate is a hyperparameter that controls how big of a step the optimizer takes in that direction.
    * Iteration: This entire process (forward pass, calculate loss, backpropagate, update weights/biases) is repeated thou

### Why are Neural Networks Powerful?

* Non-linearity: They can learn and model highly complex, non-linear relationships in data that simpler models cannot.
* Feature Learning: Especially deep networks can automatically learn relevant features from raw data, eliminating the need
* Scalability: They perform very well with large amounts of data.

### Common Types of Neural Networks:

* Feedforward Neural Networks (FNNs) / Multi-Layer Perceptrons (MLPs): The basic type, where information flows in one dire
* Convolutional Neural Networks (CNNs): Excellent for image and video processing tasks. They use "convolutional filters" t
* Recurrent Neural Networks (RNNs): Designed for sequential data (like text, speech, time series). They have "memory" that
* Transformers: A more recent architecture that has revolutionized Natural Language Processing (NLP). They use "attention

In essence, a neural network is a sophisticated function approximator that learns to map inputs to outputs by repeatedly adjusti

```

2. Few-Shot Prompting

```

print('\n=== FEW-SHOT ===')
prompt="""Explain the following in simple terms:Example:Concept: Machine LearningExplanation: Machine learning lets computers l
print('Prompt:',prompt)
resp=model.generate_content(prompt) # Generate content based on the few-shot example
print(resp.text)

```

```
=== FEW-SHOT ===
```

```
Prompt: Explain the following in simple terms:Example:Concept: Machine LearningExplanation: Machine learning lets computers learn
```

```
Concept: Neural Networks
```

```
Explanation: Neural networks let computers learn in a brain-like way, using interconnected layers to find patterns in data.
```

3. Role-Based Prompting

```
print('\n=== ROLE-BASED ===')
prompt=f"Act as a university professor. Explain Neural Networks to first-year engineering students." # Define the role-based prompt
print('Prompt:',prompt)
resp=model.generate_content(prompt) # Generate content with the specified persona/role
print(resp.text)
```

✓ 4. Chain-of-Thought Prompting

```
print('\n=== CHAIN-OF-THOUGHT ===')
prompt="""Explain Neural Networks step-by-step:1. Start with intuition2. Define components3. Explain how they learn4. Give a rea
print('Prompt:',prompt)
resp=model.generate_content(prompt) # Generate content by following the given steps
print(resp.text)
```

=== ROLE-BASED ===

Prompt: Act as a university professor. Explain Neural Networks to first-year engineering students.

Good morning, everyone! Settle in, grab your notebooks. Today, we're diving into one of the most exciting and transformative tec

Now, I know some of you might have heard the term "AI" thrown around a lot - in movies, news articles, even your smartphones. Ne

Don't worry, we're not going to turn you into neuroscientists overnight. While inspired by the human brain, artificial neural ne

The Inspiration: Our Amazing Brain

Let's start with the analogy. Think about your own brain. It's an incredible pattern-matching machine. How do you recognize a fr

Each of these biological neurons receives signals from other neurons, processes them, and then, if the combined signal is strong

Artificial Neural Networks try to mimic this *basic principle* - not the biology precisely, but the idea of interconnected proce

The Basic Building Block: The Artificial Neuron

Let's strip it down to its simplest component: the **artificial neuron**, often called a **perceptron**.

Imagine you're trying to decide if you should go for a run today. What factors do you consider?

1. **Temperature:** Is it warm enough? (Input 1)
2. **Time of day:** Do I have enough time? (Input 2)
3. **Energy level:** Am I feeling up to it? (Input 3)

An artificial neuron works similarly:

- * **Inputs (x):** It receives several pieces of information or data points. Think of these as numbers.
- * **Weights (w):** Each input is assigned a "weight." This weight represents how important that particular input is to the neu
- * **Summation:** The neuron multiplies each input by its corresponding weight and then sums all these weighted inputs together
 - * **Mathematical representation:** $(x_1 * w_1) + (x_2 * w_2) + (x_3 * w_3) + \dots$
- * **Bias (b):** We also add a "bias" term. Think of the bias as an extra "knob" that allows the neuron to make an output even
- * **Mathematical representation with bias:** $(x_1 * w_1) + (x_2 * w_2) + (x_3 * w_3) + \dots + b$
- * **Activation Function (f):** Finally, this sum passes through an "activation function." This function decides whether the ne
 - * **Simple example:** A step function: if the sum is above a certain threshold, output 1 (go for a run!); otherwise, output
 - * **More complex (but common) examples:** Sigmoid, ReLU (Rectified Linear Unit), Tanh. These produce a continuous output, no

So, the output of a single neuron is essentially: $\text{Output} = f((x_1 * w_1) + (x_2 * w_2) + \dots + b)$

Connecting Neurons: The Network

A single neuron is fairly limited. The real power comes when you connect many of them together into a **network** of layers:

1. **Input Layer:** This is where your raw data enters the network. If you're classifying images, each input neuron might repre
2. **Hidden Layers:** These are the "thinking" layers. Each neuron in a hidden layer receives inputs from *all* the neurons in
 - * Neurons in earlier hidden layers might learn to detect simple features (like edges or corners in an image).
 - * Neurons in later hidden layers combine these simple features to detect more complex patterns (like shapes, parts of obje
3. **Output Layer:** This layer gives you the final result or prediction of the network.
 - * For classifying "cat" or "dog," it might have two output neurons.
 - * For predicting a house price, it might have a single output neuron with a numerical value.

Each connection between neurons has its own **weight**, and each neuron has its own **bias**. It's these millions (or even billi

How Does it Learn? The Training Process

This is where the magic (and the math!) happens.

1. **Training Data:** You feed the network a vast amount of **labeled data**. For example, thousands of images, each clearly la
2. **Initial Guess:** Initially, all the weights and biases in the network are set randomly. So, the network is completely clue
3. **Forward Pass:** You take an input (e.g., an image of a cat) and pass it through the network, layer by layer, until it prod
4. **Calculate Error (Loss):** You then compare the network's prediction to the **actual** label (the "ground truth"). The differ
5. **Backpropagation & Optimization (Gradient Descent):** This is the core learning algorithm.
 - * The network essentially calculates *how much* each weight and bias in the network contributed to that error.
 - * It then adjusts those weights and biases *slightly* to reduce the error. Think of it like tuning a complex instrument wi
 - * This adjustment process is guided by calculus (specifically, gradients, which tell you the direction of steepest ascent/
6. **Iteration:** You repeat steps 3-5 millions of times, using many different training examples. Over time, the network gradua

Why Are They So Powerful?

- * **Feature Learning:** Unlike traditional programming where *you* have to define exactly what features to look for (e.g., "a
- * **Non-Linearity:** Thanks to activation functions, they can model incredibly complex, non-linear relationships in data that
- * **Scalability:** With enough data and computational power, they can achieve astonishing performance on tasks previously cons

Where Are They Used?

You interact with Neural Networks every single day:

- * **Image & Video Recognition:** Face ID on your phone, self-driving cars recognizing pedestrians and traffic signs, medical i
- * **Natural Language Processing (NLP):** Virtual assistants (Siri, Alexa), translation services (Google Translate), spam filte
- * **Recommendation Systems:** Netflix suggesting movies, Amazon recommending products.
- * **Robotics & Control:** Learning to grasp objects, navigate complex environments.
- * **Scientific Research:** Drug discovery, climate modeling, particle physics.

```
### A Word of Caution for Engineers
```

```
As engineers, it's crucial to also understand their limitations:
```

```
Prompt: Explain Neural Networks step-by-step:1. Start with intuition2. Define components3. Explain how they learn4. Give a real-Neural Network Example: A hypothetical agent, equipped with AI capabilities, is tasked with daily tasks from a user. It interacts with various assistants to recommendation engine
```

```
* **Computational Cost:** Training large neural networks can be incredibly expensive in terms of computing power and time.
```

```
--- **Black Box Problem:** It can be very difficult to understand *why* a neural network made a particular decision. This "lack
```

```
* **Bias:** If your training data contains biases (e.g., more images of one demographic than another), the neural network will
```

```
### 1. Intuition: A Team of Experts Making Decisions
```

```
### Conclusion
```

```
Imagine you want to decide if you should *buy a house*. You wouldn't just look at one factor; you'd consider many:
```

```
So, that's a whirlwind tour of Neural Networks. They are powerful, adaptable, and a cornerstone of modern AI. They represent a f
```

```
* **Location:** Is it in a good neighborhood?
```

```
As future engineers, it's important to understand these concepts is vital. Whether you end up working in civil, mechanical, electrical, or soft
```

```
* **Size:** Is it big enough for your family?
```

```
Keep in mind, this is just a simplified overview. The real world of AI is much more complex. But as you learn, you'll be well on your way.
```

```
* **Schools:** Are there good schools nearby?
```

```
Any questions?
```

```
Each of these factors contributes to your final decision. Some factors might be *more important* to you (e.g., location might
```

5. Self-Critique / Revision Prompting

A Neural Network works similarly. It's like a *team of tiny interconnected "decision-makers" (neurons)*. Each neuron takes som

The magic is that these "decision-makers" don't start knowing what's important. They *learn* through trial and error, adjustin

```
print('\n=== SELF-CRITIQUE ===')
```

```
prompt=f"""Explain Neural Networks in 5 sentences.Then:- Identify confusing parts- Rewrite them more clearly""" # Define the se
```

```
print('Prompt:',prompt)
```

```
resp=model.generate_content(prompt) # Generate content and then critique/revise it
```

```
print(resp.text)
```

```
1.  **Neurons (Nodes):**
```

```
* **Analogy:** A single "decision-maker" or "processing unit."
```

```
=== SELF-CRITIQUE does:** Receives input signals, performs a simple calculation, and then sends an output signal.
```

```
Prompt: Explain Neural Networks in 5 sentences.Then:- Identify confusing parts- Rewrite them more clearly
```

```
Here's an explanation of the basic components of a Neural Network in 5 sentences:
```

```
* Each input is multiplied by a *weight*.
```

```
Neural Networks are a simplified version of the human brain's structure. They consist of artificial "neurons" (no
```

```
* A *bias* value is added to the sum.
```

```
--- * This final sum passes through an *activation function* to produce the neuron's output.
```

```
### Identifying Confusing Parts & Rewriting Them
```

```
* Neurons are organized into layers.
```

```
**Confusing Part 1:** Computational models inspired by the human brain.**
```

```
* **Why it's confusing:** The "inspired" part is vague. What exactly is being inspired? about the brain are they inspired by, beyond just a general
```

```
* **Rewritten more clearly:** "Recreate a network that mimics the flow of information in the human brain, from input to output, through hidden layers." Recreate a network that mimics the flow of information in the human brain, from input to output, through hidden layers.
```

```
* **Hidden Layers:**
```

```
**Confusing Part 2:** They process information in a way that is not directly observable, allowing for complex patterns to be learned.
```

```
* **Why it's confusing:** "Not directly observable" is confusing. It's like saying "the magic happens in the black box." They process information in a way that is not directly observable, allowing for complex patterns to be learned.
```

```
* **Rewritten more clearly:** "They consist of artificial 'neurons' (nodes) arranged in layers, which *receive* and perform sim
```

```
* **Analogy:** The "final decision" or "answer."
```

```
**Confusing Part 3:** It does a weighted sum of the inputs, where the weights are adjusted during training.
```

```
* **Why it's confusing:** "Weighted sum" is a mathematical term that might be confusing. How does the network "adjust" weights, and what's the go
```

```
* **Rewritten more clearly:** "These nodes are connected by 'weights,' which *represent* the importance of each connection to t
```

```
* **Analogy:** Wires or pathways that connect neurons between layers.
```

```
**Confusing Part 4:** It takes in new, unseen data and makes predictions or classifications.
```

```
* **Why it's confusing:** "Unseen data" is a bit abstract. What are some concrete examples of predictions or classifications?
```

```
4. **Rewritten more clearly:** "This allows the trained network to *identify* complex relationships in data, such as distinguish
```

```
* **Analogy:** The "importance knob" for each connection.
```

```
--- * **What it does:** A numerical value associated with each connection. It determines how much influence the input from one
```

```
### Revised Explanation (using the clearer parts):
```

```
* **Analogy:** A "tendency to fire" or an "offset."
```

```
Neural Networks are a simplified version of the human brain's structure. They consist of artificial "neurons" (no
```

```
6.  **Activation Function:**
```

```
* **Analogy:** A "switch" or a "decision rule" within each neuron.
```

```
* **What it does:** A mathematical function applied to the weighted sum + bias. It introduces non-linearity into the netwo
```

```
* **Examples:**
```

```
* **ReLU (Rectified Linear Unit):** If the input is positive, output the input. If negative, output zero. `f(x) = max(
```

```
print('\n=== FORMAT CONTROL ===')
```

```
prompt="""Explain Neural Networks with constraints:Audience: 10-year-oldTone: FriendlyFormat: Bullet pointsLength: 6 bullets ma
```

```
print('Prompt:',prompt)
```

```
resp=model.generate_content(prompt) # Generate content adhering to the specified style and format
```

```
print(resp.text)
```

```
1.  **Initial Guess (Forward Propagation):**
```

```
* The network starts with *randomly* initialized weights and biases*. It literally has no idea what it's doing.
```

```
* You feed it a piece of *training data* (e.g., an image of a cat along with the label "cat").
```

```
* The input data travels *forward* through the network, from the input layer, through the hidden layers, to the output 1
```

```
* Each neuron performs its calculation (weighted sum + bias, then activation).
```

```
* The output layer produces an *initial prediction* (e.g., for the cat image, it might randomly guess "70% dog, 30% cat")
```

```
2.  **Checking the Guess (Loss Function):**
```

```

    After the initial prediction, the network compares its prediction to the "actual correct answer" (the label we provide).
    * A **Loss Function** (or Cost Function) calculates how "wrong" the prediction was. It quantifies the error. A higher loss ==
    **FORMAT: ANALOGY** If you guess a house price of $500,000 and the actual price was $400,000, your loss function would tell you
    Prompt: Explain Neural Networks with constraints: Audience: 10-year-old Tone: Friendly Format: Bullet points Length: 6 bullets max In
    Here's a question for you: Explain the concept of a gradient descent.
    * This is the core of learning. The network needs to figure out *how* to adjust its weights and biases to reduce the error
    * An image is fed into the network. The network's job is to identify the image. It does this by passing the image through the network, from the output
    * Instead of just saying "cat" or "dog", it can be more specific. For example, it might say "a Siamese cat" or "a golden retriever". Weigh the different parts of the image that contributed to the overall error
    * This is the core of learning. The network needs to figure out *how* to adjust its weights and biases to reduce the error
    * When it makes a guess, it also calculates the error. This error is used to adjust the weights and biases. The network then repeats this process many times, improving its accuracy.
    * After several iterations, the network's predictions become more accurate. It can now identify new, unseen images with a high degree of accuracy.
    * These clever techniques help the network learn from its mistakes and become a more accurate predictor.
    * **Analogy:** You're tuning a guitar. You pluck a string (forward propagation), hear it's out of tune (loss function), and

```

4. **Repeat, Refine, Learn (Training Epochs):**

✓ Bonus — Multilingual Prompting

```

Steps 1-3 are repeated thousands, millions, or even billions of times with different pieces of training data.
* Each full pass through the entire training dataset is called an **epoch**.
* With each iteration, the weights and biases are incrementally adjusted, making the network's predictions more and more a

```

```

print('\n=== MULTILINGUAL (TAMIL) ===')
prompt="Explain Neural Networks in Tamil in 4 simple sentences." # Define the prompt to generate content in a specific language
print('Prompt:',prompt)
resp=model.generate_content(prompt) # Generate content in Tamil
print(resp.text)

```

How a Neural Network Solves It:

```

=== MULTILINGUAL (TAMIL) ===
1. **Input:** Prompt: Explain Neural Networks in Tamil in 4 simple sentences.
    An image is fed into the network.
    நாம் பியல் வலைப் பிள்ளைகள் (Neural Networks) மனித மூளை எப்படி செயல்படுகிறதோ அதைப் போலவே கற்றுக்கொள்ளும்.
    * The image is represented as a grid of numbers (pixels). Each pixel's color and intensity becomes an input feature to the

```

2. **Network Architecture:**

✓ Bonus — Length/Compression Control

```

* **Input Layer:** Neurons corresponding to each pixel value.
* **Hidden Layers:** Multiple hidden layers. These layers learn to identify progressively more complex features.
* **First Hidden Layer:** Might learn to detect simple edges, lines, and corners.
* **Next Hidden Layer:** Might combine those edges to detect shapes like ears, eyes, or whiskers.

```

```

print('\n=== LENGTH CONTROL ===')
prompt="Explain Neural Networks in exactly:(1) One sentence(2) One paragraph(3) One tweet(4) One analogy(5) One exam definition"
print('Prompt:',prompt)
resp=model.generate_content(prompt) # Generate content with precise length control
print(resp.text)

```

```

    After calculation, the loss function calculates how far off the guess is from the correct label (e.g., 0.007).
    * **Adjustment (Backpropagation):** The error is backpropagated. The network figures out which weights contributed to the
    == LENGTH CONTROL == This process is repeated millions of times with countless cat and dog images. Over time, the network's we
    Prompt: Explain Neural Networks in exactly:(1) One sentence(2) One paragraph(3) One tweet(4) One analogy(5) One exam definition
    Here's a question for you: Explain the concept of a gradient descent.
    * Once trained, you can show the network a **new, unseen image**.

```

1. **On the way, the pixels flow through the learned network.

A neural network is a computational system modeled loosely on the human brain's structure and function, designed to identify patterns in data. It learns from data to perform tasks like image recognition, language translation, and recommendation systems.

3. **One tweet:**

Neural Networks are AI systems inspired by the human brain. They learn patterns from data through interconnected "neurons" a

4. **One analogy:**

Imagine a neural network as a committee of interconnected "decision-makers" (neurons) organized into teams (layers). Raw inf

5. **One exam definition (50 words):**

A neural network is a computational model inspired by the brain, consisting of interconnected nodes (neurons) in layers. It

Reflection Questions for Students

- Which prompt produced the clearest output?
- Which prompt produced the most structured output?
- How did the role affect the tone?
- Did reasoning steps increase clarity?
- Why are constraints powerful for control?
- Which output works best for learning?

